

Animal-Themed Clock Radio

ECE 299 Final Report

University of Victoria

Group ID: G11

Garrett Ma - V00976948

Karl Hilario – V00983970

Table of Contents

Acknowledgement	3
Abstract	3
List of Tables.....	4
List of Figures	4
Project Goal.....	4
Constraints	4
Project Expectations	5
Design Choices.....	5
Project Planning.....	8
Circuit Design and Schematic	10
Bill of Materials	11
Printed Circuit Board	12
Enclosure Design	15
Testing & Validation	17
Code of Ethics (EGBC).....	17
Conclusion & Recommendations.....	17
References	19
Appendix A	19

Acknowledgement

We acknowledge with respect the Lekwungen peoples on whose traditional territory the university stands and the Songhees, Esquimalt and WSÁNEĆ peoples whose historical relationships with the land continue to this day.

Abstract

This project presents the design and implementation of an animal themed clock radio. The clock radio should be able to play the radio, show the time and other crucial clock information, and be user friendly. The clock radio will be controlled by Raspberry Pi Pico W using the MicroPython programming language and control other components using the SPI protocol.

A design for our PCB was breadboarded, drawn out and routed in KiCad, then manufactured at a facility. In our case the PCB got a little cramped, so we used a 4-layer PCB design which aided in routing simplicity. Some mistakes in the schematic and component placement caused us to rework our board but worked flawlessly afterwards. The housing was designed in Solidworks, had a simple and functional design, and was complimented with a cute cat design.

Overall, the core functionality of the radio was completed for both hardware and software. Some PCB rework was required, to get the screen to work and oriented in the correct way. Additionally, the reception in the lab was bad and our radio data system (RDS) information would not load correctly.

List of Tables

Table 1: Table showing steps and roles in accomplishing project expectations	9
Table 2: Table showing the costs of each component.....	11

List of Figures

Figure 1: Circuit Schematic	10
Figure 2: Top side of PCB.....	12
Figure 3: Bottom side of PCB.....	13
Figure 4: 3D view of populated PCB	14
Figure 5: 3D model of enclosure design	16
Figure 6: Drawing of enclosure design.....	16
Figure 7: Picture of enclosure in slicer which shows standoffs that weren't printed	17

Project Goal

The goal of this project is to develop and manufacture a clock radio controlled by a Raspberry Pi Pico W programmed with MicroPython. The alarm shall integrate core functionality like, time tracking in 12 and 24 hour time, alarm, and radio. All this information will be displayed on a 1.3" OLED SPI-based display, audio will be played through an integrated speaker, and 2 buttons and 2 rotary encoders will be included to interact with the clock radio.

The clock radio will be designed by the student and some components will be sent for manufacturing. We will design a PCB which will hold and connect all the components. A housing will also be designed with an animal theme without compromising on the functionality of the radio. In the end, the completed radio will be demonstrated and a conversation on the design and ideas behind the project will be shown.

Constraints

There are some constraints on the project to encourage creativity and test knowledge. Since the project cannot be a breadboard project, money will have to be spent on components, PCB manufacturing, and the housing. Therefore, our goal is to minimize cost and mistakes. Additionally, we are limited to one Raspberry Pi Pico W and it must control all the components and communications and programmed with MicroPython.

Project Expectations

1. Design and develop a prototype of an animal themed clock-radio that has the following functionalities.

Clock functionalities

- Set/Edit time through a user interface
- Set/Edit alarm time through a user interface
- Display time in 12-hour/24-hour format as desired by the user
- Trigger alarm as set by the user
- Snooze alarm & turn-off alarm through a user interface

Radio functionalities

- Tune to at least one FM radio channel through a user interface
 - Play radio output on to a speaker
 - Volume control of audio output through a user interface
 - Display radio channel info to the user
2. Design a two-layer printed circuit board (PCB) for the clock-radio and get it manufactured from a commercial PCB manufacturer.
 3. Solder the components on to the manufactured PCB.
 4. Design and manufacture an animal themed enclosure for the clock-radio.
 5. Present the clock-radio packaged in an animal themed enclosure by the last week of labs.

Design Choices

Clock Functionalities

- Set/Edit Time through UI

Design: Time was managed using the onboard RTC module (machine.RTC) from the RP2040. Two rotary encoders with push-button functionality allow users to set hours, minutes, and alarm time via menu-based UI.

Justification: The Raspberry Pi Pico W supports flexible GPIO assignment and RTC libraries in MicroPython. Rotary encoders with built-in button presses simplify interface logic while reducing hardware clutter.

- Alarm and Snooze

Design: Alarms were handled by checking the current time each second. If matched, a snooze/cancel OLED interface appears.

Justification: A state-driven alarm logic was implemented in radio.py. Alarms trigger audio playback through the FM module with the volume pre-set to 15. Snooze delays are tracked using time.time().

- 12/24 Hour Format Switching

Design: A toggle stored in is_24_hour_format determines how the time is rendered on the OLED screen.

Justification: Using MicroPython's datetime formatting tools, this toggle allows flexible display formatting without increasing computational complexity.

Radio Functionalities

- FM Tuning and Playback

Design: The RDA5807M FM chip is controlled via I2C using rotary encoder input. Frequency values are mapped to 10-bit channel values per the datasheet.

Justification: RDA5807M supports fast tuning via I2C and has built-in mute, bass boost, and stereo options. Rotary encoder input is intuitive and allows for rapid tuning increments.

- Volume Control

Design: One encoder adjusts the volume from 0 to 15. The corresponding register (0x05) is updated via I2C with each change.

Justification: Volume is handled entirely in the digital domain, reducing component count and avoiding analog noise pickup.

- RDS Display

Design: Station metadata (name, song info) is decoded via the GetRDS() function in radio.py, parsed into scrollable text and shown on the OLED.

Justification: RDA5807M supports RDS standard decoding through registers that provide 4-block group messages. The implementation handles character pairing, filtering, and display buffering.

OLED Display and UI

- OLED (SSD1306 SPI)

Design: Display initialized using ssd1306.py with SPI interface to show current time, frequency, volume, RDS data, and alarm prompts.

Justification: SPI offers faster transfer and fewer glitches compared to I2C. The

SSD1306 controller supports page addressing and contrast adjustment, handled through command macros in the driver.

- Input Devices

Design: Two rotary encoders are used—one for volume and one for frequency—with both having push-button functionality. Two additional physical push buttons are used for alarm and time format toggle.

Justification: Minimizing input hardware while maximizing function was a design goal. Rotary encoders with built-in buttons enabled 4 total inputs using only 2 encoders and 2 extra buttons.

PCB Design and Manufacturing

- 4-layer PCB

Design: Designed in KiCad with proper power/ground planes for analog and digital domains. High-speed I2C lines, speaker traces, and Pico W were routed on different layers.

Justification: A 4-layer board improves signal integrity, reduces EMI, and allows better separation between digital control and analog audio. This choice also facilitated a compact layout within the enclosure.

- Component Soldering

Design: A combination of SMD and through-hole components were manually soldered.

Justification: Through-hole parts allowed reliable mounting for Pico W and connectors. SMD resistors and caps helped with compact layout. PCB assembly was validated with test points and oscilloscope probing.

Enclosure and Packaging

- 3D-Printed Animal-Themed Housing

Design: Enclosure modelled in SolidWorks and printed in PLA with snap-fit supports for the PCB and faceplate openings for UI controls. Cat-themed housing was designed.

Justification: PLA is lightweight and thermally stable for electronics. SolidWorks enabled precise fit and integration of design aesthetics with functional constraints.

Project Planning

To complete the animal-themed clock radio within the project timeline, we outlined the steps needed to meet each of the course expectations and coordinated team roles accordingly. While we initially divided tasks between hardware and software, in practice, both team members worked collaboratively on all aspects of the project—including debugging, testing, and integration.

The table below summarizes how we addressed each project expectation, the major tasks involved, and the shared roles of both team members in completing them.

Team Roles and Responsibilities

Project Expectation	Steps Taken	Team Roles
1. Functional Clock System	• Designed menu-based UI for setting time and alarm • Implemented 12/24-hour formatting toggle • Alarm and snooze logic programmed and tested	- Garrett developed alarm trigger and snooze code - Karl created UI menus for time setting and formatting
2. Functional FM Radio	• Interfaced RDA5807M via I2C for tuning and RDS • Connected to LM386 amplifier and speaker • Volume and channel control through encoders	- Garrett handled radio driver and I2C integration - Karl tested tuning, audio output, and OLED feedback
3. 4-layer PCB Design & Manufacturing	• Designed schematic and layout in KiCad • Added power, analog, and digital signal traces across four layers • Sent for fabrication and soldered components	- Karl led schematic/PCB design - Garrett assisted with debugging and soldering components

4. Animal-Themed Enclosure	• Modelled in SolidWorks with cutouts for screen, buttons, and speaker • 3D printed and assembled enclosure with mounted board and display	- Karl designed the enclosure - Garrett printed the enclosure and tested PCB/enclosure alignment and mounting
5. Full Integration & Packaging	• Assembled all components • Final OLED UI implemented • Validated alarm, radio, and display functionality inside enclosure	- Both members tested and validated features jointly - UI polish and bug fixing done collaboratively

Table 1: Table showing steps and roles in accomplishing project expectations

Execution vs. Original Plan

We followed the original project milestone plan closely, with all major deliverables completed on schedule. These include:

- Breadboard testing and validation of all subsystems
- Completion and assembly of a custom 4-layer PCB
- Final 3D-printed themed enclosure
- Integrated software combining timekeeping, FM radio, alarm, and UI display

While the roles were originally divided by hardware and software focus, in practice both team members worked collaboratively on all parts of the project. Tasks such as debugging, integration, and testing were conducted together. This teamwork helped ensure rapid troubleshooting and polished results.

Circuit Design and Schematic

Below in figure 1 is the circuit schematic drawn in KiCad. Each of these components were iteratively added after each tutorial after learning how each piece worked. Then after breadboarding everything together we would add more components and wires onto the schematic after testing.

Two big points are the resistors and capacitors we added to debounce and reduce noise of certain components. Firstly, power from a USB is often noisy and sometimes not grounded very well. Adding two capacitors from the 3.3v and 5v rails in parallel with ground help smooth out the output of the power leading to a better sounding speaker and better performing op-amp. Secondly, we had issues with debouncing the rotary encoders and it would randomly output different values then the direction we were turning it. Adding a resistor in series helped limit current which helps reduce the transient spikes that the GPIO pin would read. Additionally, software tricks like adding a finite state machine (FSM) helped ensure we didn't have unwanted values. All of this led to the actual 4-layer PCB found in figure 2.

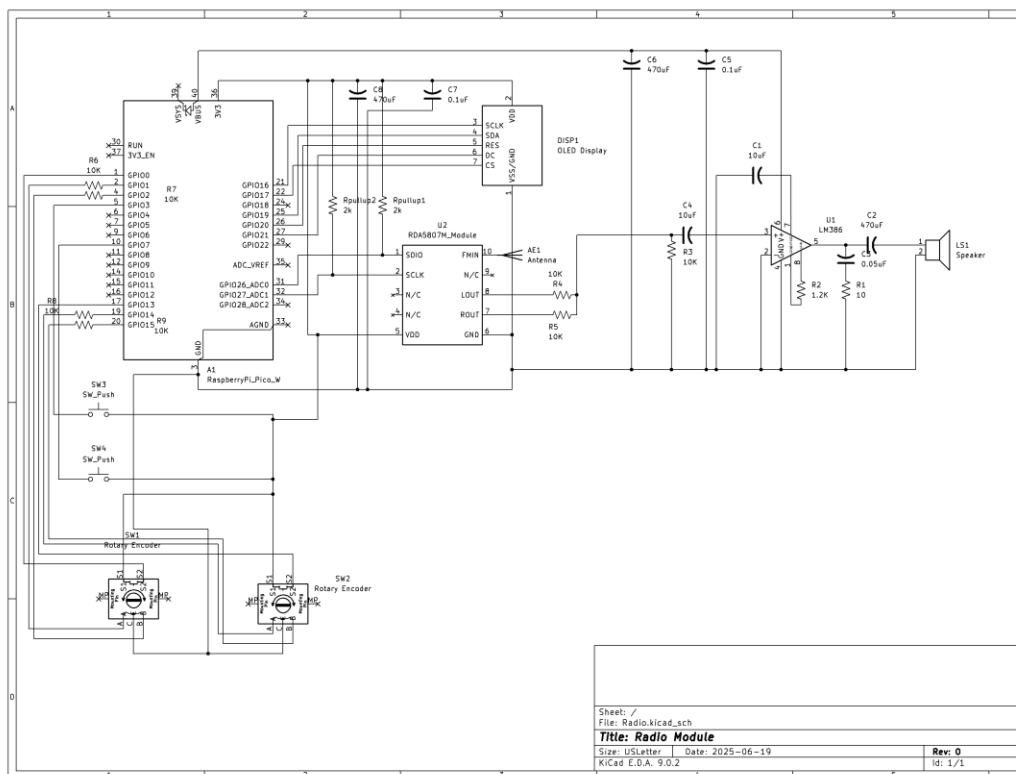


Figure 1: Circuit Schematic

Bill of Materials

The table below outlines the Bill of Materials (BOM), listing all major components used in the final design along with their specifications, quantities, and sources.

Label in schematic	Component Description	Cost/unit quantity	Source of cost information
A1	Raspberry Pico W	\$10.00 x 1	ECE 299 Parts List with price info
PCB	Manufactured PCB	\$10.00 x 5	PCBWay invoice
U1	LM386N-4 Audio Amplifier	\$1.50 x 1	ECE 299 Parts List with price info
U2	RDA5807M on Through-hole PCB	\$2.00 x 1	ECE 299 Parts List with price info
DISP1	1.3" OLED Display SPI	\$4.00 x 1	ECE 299 Parts List with price info
LS1	Speaker 2W 8Ω	\$4.00 x 1	ECE 299 Parts List with price info
SW1, SW2	Rotary Encoder	\$1.00 x 2	ECE 299 Parts List with price info
SW3, SW4	Pushbutton Microswitch PCB	\$0.25 x 2	ECE 299 Parts List with price info
R1, ..., R8, Rpullup1, Rpullup2	Resistors	Free	ECE 299 Parts List with price info
C1, ..., C8	Capacitors	Free	ECE 299 Parts List with price info

Table 2: Table showing the costs of each component

Printed Circuit Board

In the 3 pictures below are the empty PCB and a fully populated PCB. Like above, we had used a 4-layer PCB to separate all the signals, power and ground into different layers and kept the PCB very compact. Each side is 75mm, giving us an area of 5,625 mm² which is above the 3000mm² expectation.

A few problems we ran into after ordering the PCB, lots requiring ingenuity and handywork. We realized first that the orientation of the Pico had the USB port facing inwards meaning that any of the components in that direction had to be mounted on the back (C1, C2, etc.) This led to us realizing that we should have just put everything on the back and only left the screen and buttons on the front which would have let us have it nicely pressed up against the front of the housing rather than it just laying there. After assembling the board, the screen wouldn't turn on and it was upside down. Looking at the schematic, all of the OLED pins were off by one and as a result we had to rework the board. After reworking the board with longer wires which gave us room to flip the display it finally worked.

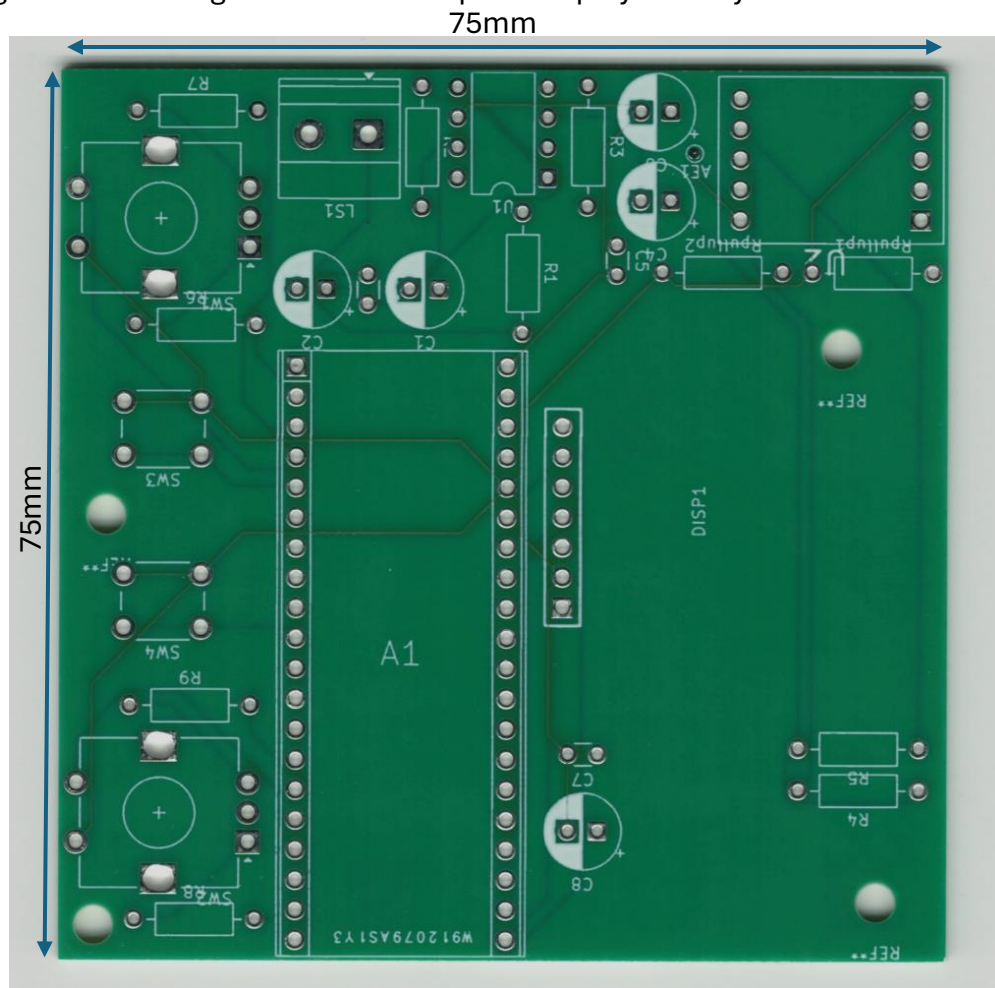


Figure 2: Top side of PCB

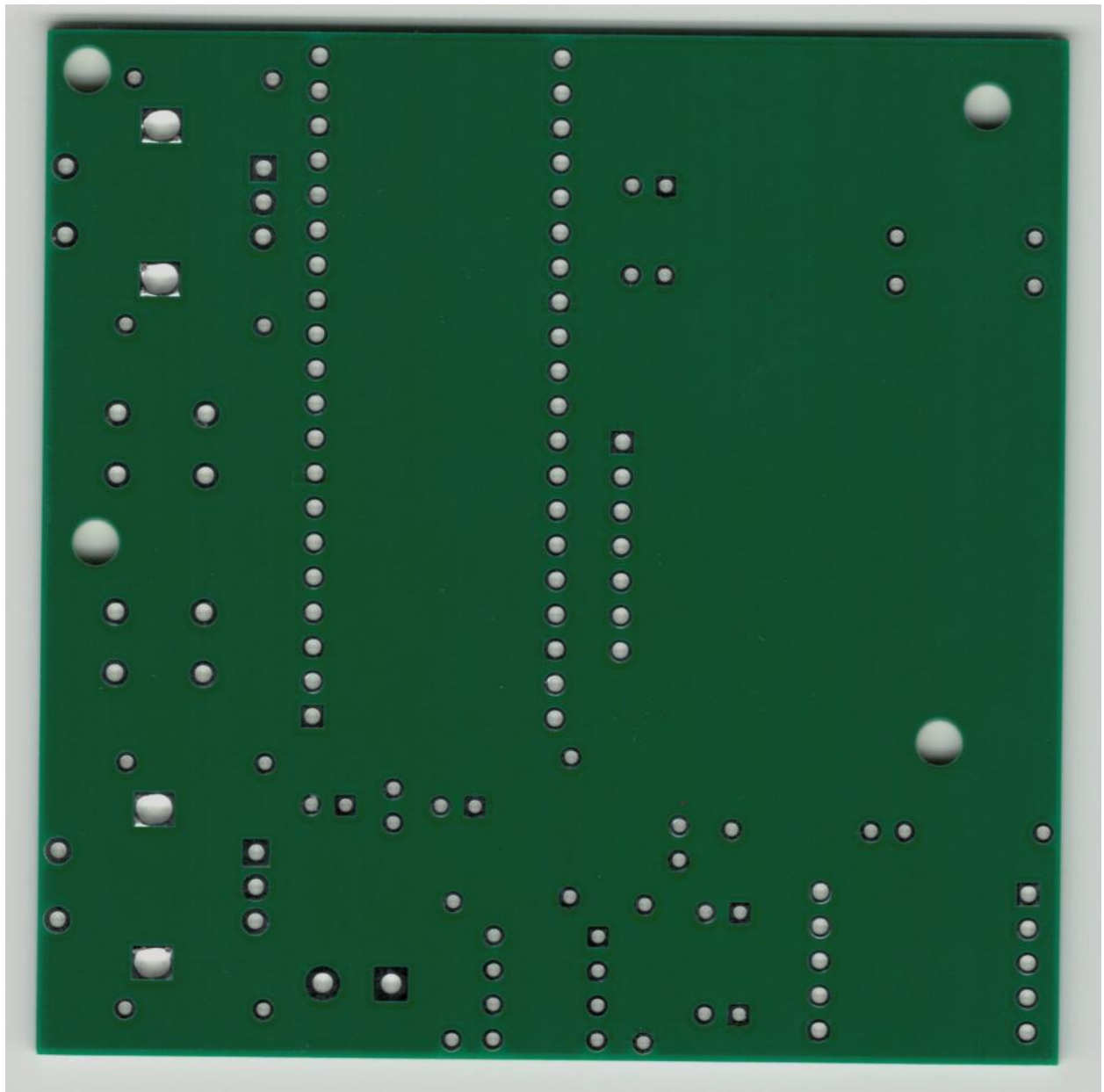


Figure 3: Bottom side of PCB

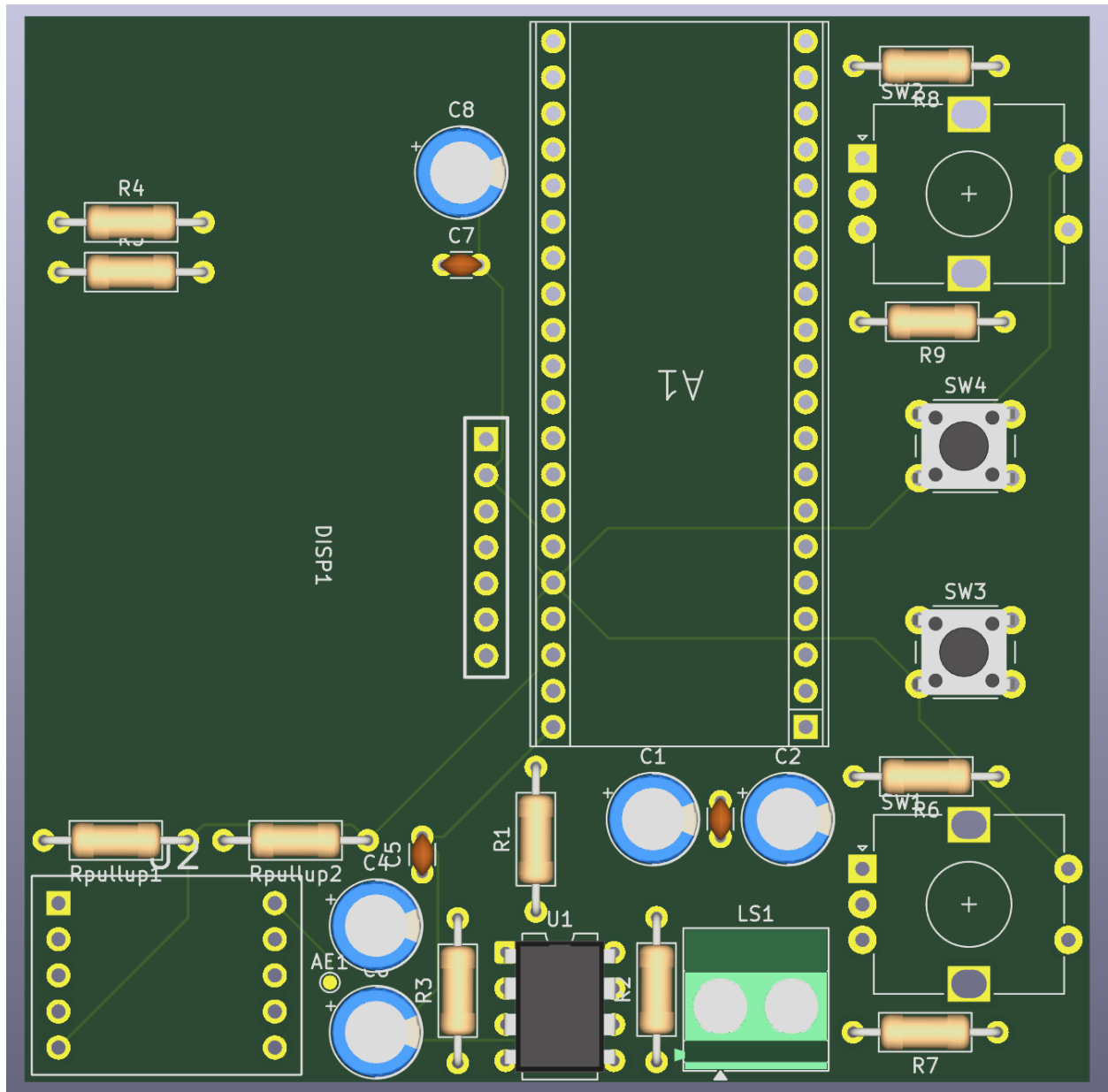


Figure 4: 3D view of populated PCB

Enclosure Design

The enclosure was modelled using SolidWorks to match the project's animal-themed aesthetic while maintaining functional requirements for component access and airflow. The design features precise cutouts for the OLED display, rotary encoders, speaker grill, and USB/power access to the Raspberry Pi Pico W.

The enclosure consists of two interlocking halves that were 3D printed using PLA filament. PLA was chosen for its ease of printing, rigidity, and thermal stability, which makes it ideal for electronic housings. Tabs and standoffs were integrated into the model to securely hold the custom PCB in place and align the display with the outer face.

The front panel includes the OLED window and symmetrical placement of the encoders and speaker, providing a clean and intuitive user interface (see Fig. 5). The rear section allows routing for power and audio connections while enclosing the LM386 amplifier and speaker mount (see Fig. 6).

Standoffs and mounting tabs were integrated into the SolidWorks model to support the PCB and hold the display flush with the outer surface. However, as shown in Fig. 7, these standoffs were present in the slicer preview but were not printed in the final version due to support and clearance issues. The enclosure still functioned correctly without them, as the PCB fit snugly inside the shell and was stabilized using press-fit tolerances and hot glue reinforcement where needed.

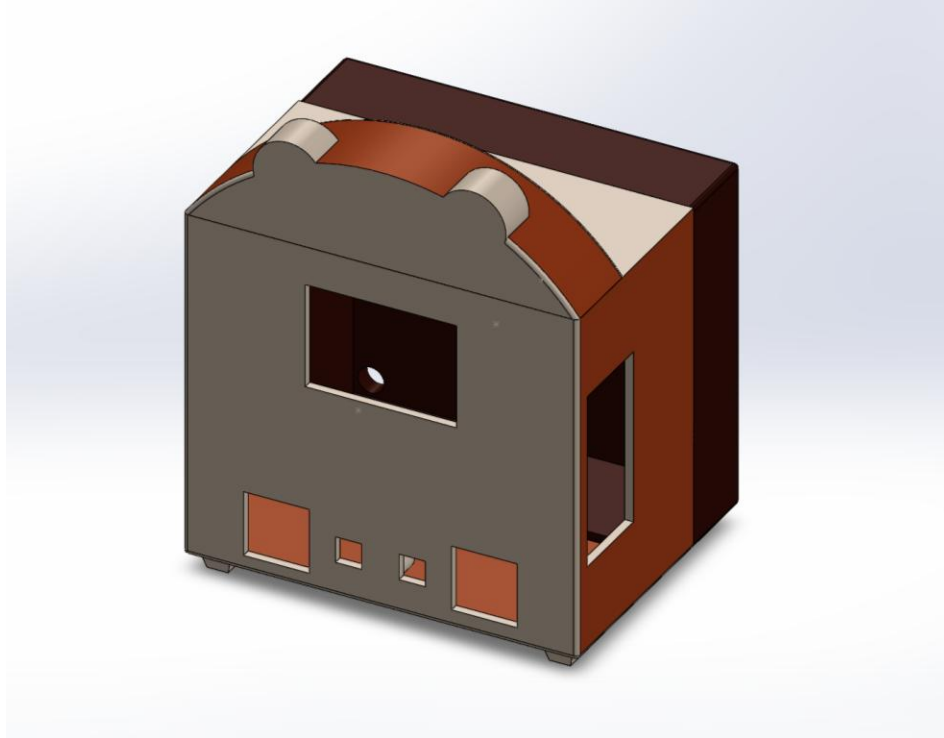


Figure 5: 3D model of enclosure design

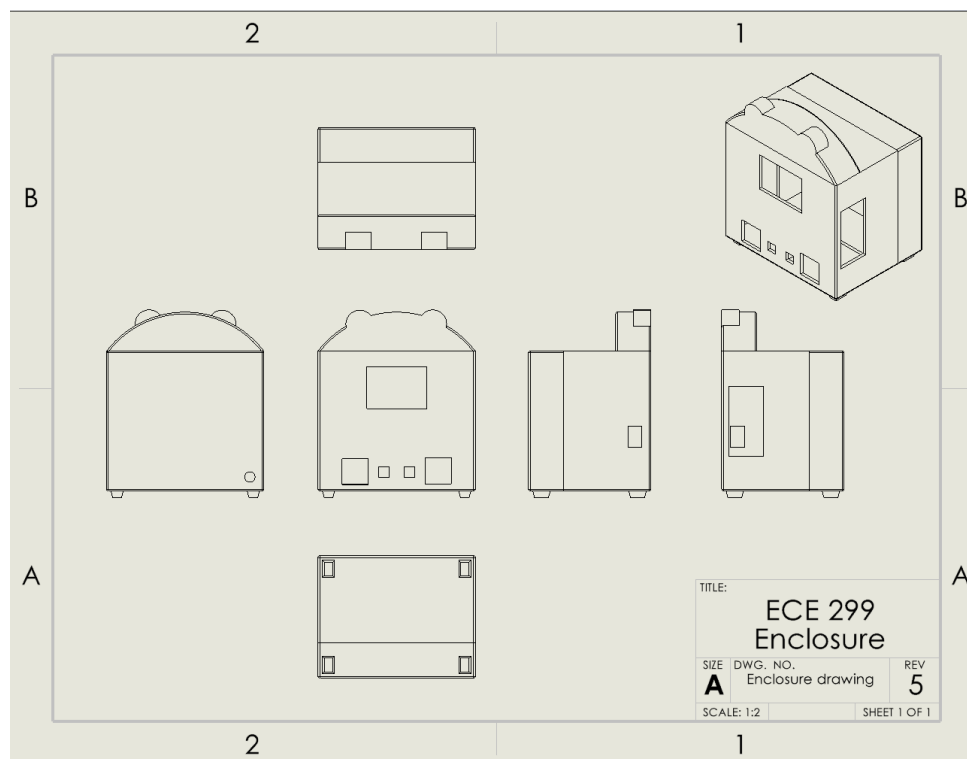


Figure 6: Drawing of enclosure design

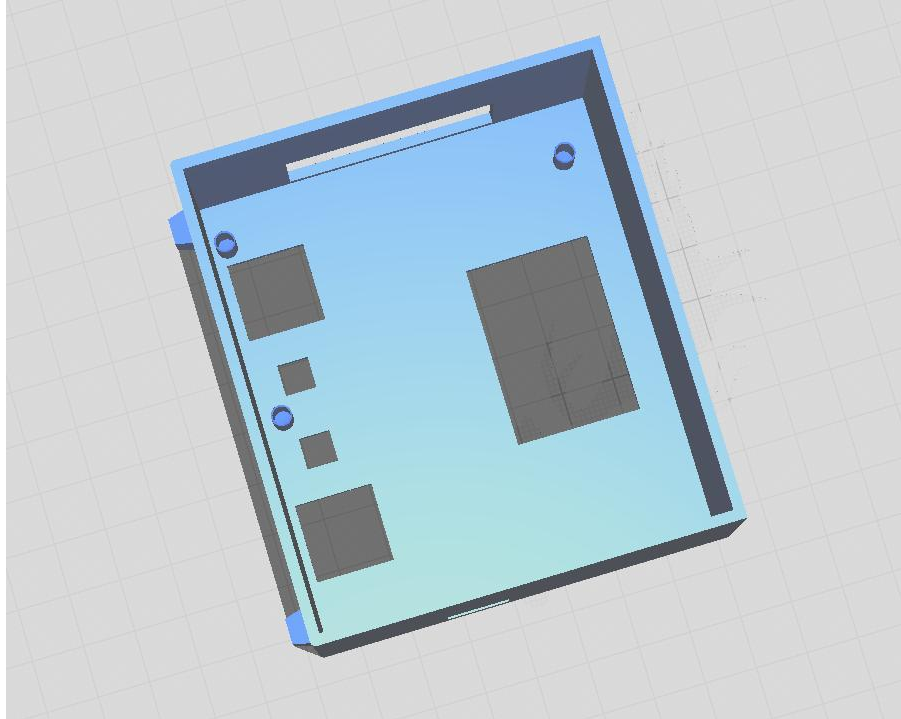


Figure 7: Picture of enclosure in slicer which shows standoffs that weren't printed

Testing & Validation

A video to the full testing of the hardware and software can be found [here](#).

Code of Ethics (EGBC)

During the development of the clock radio, we took appropriate precautions and followed EGBC's Code of Ethics [X]. This final report shows our development process which shows that we are practicing in fields where training will make us professionally competent.

Additionally, this final report provides accurate information with respect to our qualifications and experience and that we are ensuring our safety first by working on a breadboard first ensure our clock pulls a reasonable amount of power and not damage property. These are just some of the relevant Code of Ethics we followed throughout the development of the clock radio.

Conclusion & Recommendations

The development of the clock radio in this project was a fun project which allowed us to apply fundamental engineering principles and apply our knowledge from previous courses. We got to learn how to use industry standard tools such as KiCAD and SolidWorks which

we can use in our personal projects or future jobs/classes. We also got to apply our coding knowledge, circuitry, and planning skills which aided in task completion. The issues we faced during the project could have been easily avoided with additional time to review our PCB or having extra time to order a new PCB. Finally, if we had more thoroughly reviewed the rubric we would have seen that the radio was not suffice and implemented the piezo buzzer to meet the expectations. This project gave us valuable hands-on experience developing, troubleshooting, and project management.

Recommendations:

If we were to pass on any advice to future students based on our experience:

1. Read the rubric in full, list components you know you will need from the 5 tutorial labs and from the rubric to make sure you hit all expectations.
2. Send the PCB for manufacturing early. Double, triple check your schematic to ensure nothing was accidentally misplaced to reduce any reworks to your PCB which minimizes costs and potential reworking mistakes.
3. Make use of the breadboard. It made troubleshooting much easier as we already knew our design worked even before sending our PCB for production.

References

- [1] Engineers and Geoscientists British Columbia, "Code of Ethics," [Online]. Available: <https://www.egbc.ca/Complaints-Discipline/Code-of-Ethics/Code-of-Ethics>
- [2] Raspberry Pi Foundation, "Raspberry Pi Pico W Datasheet," [Online]. Available: <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>
- [3] Texas Instruments, "LM386 Low Voltage Audio Power Amplifier," [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm386.pdf>
- [4] ECE 299 Course Team, "ECE 299 Lab Manual," University of Victoria, 2024. [Online]. Available on Brightspace.
- [5] Adafruit, "SSD1306 OLED Display Driver," [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- [6] "Getting Started in KiCad," University of Victoria, 2024.
- [7] "Getting Started with SolidWorks," University of Victoria, 2024.
- [8] Analog Devices, "Basics of I2C Protocol," [Online]. Available: <https://www.analog.com/media/en/technical-documentation/application-notes/AN686.pdf>
- [9] RDA Microelectronics, "RDA5807M FM Receiver IC – Programming Guide," [Online]. Available: https://datasheet.lcsc.com/lcsc/1811141540_RDA-Microelectronics-RDA5807M_C27381.pdf

Appendix A

A link to our GitHub repository with our code and diagrams can be found [here](#).